

JMORF — Morpho-Syntax

Long Distance Dependencies

Francis Bond

Palacký University

<https://fcbond.github.io/>

`bond@ieee.org`

Lecture 11

Location: SV 2.39

JMORF (2025)

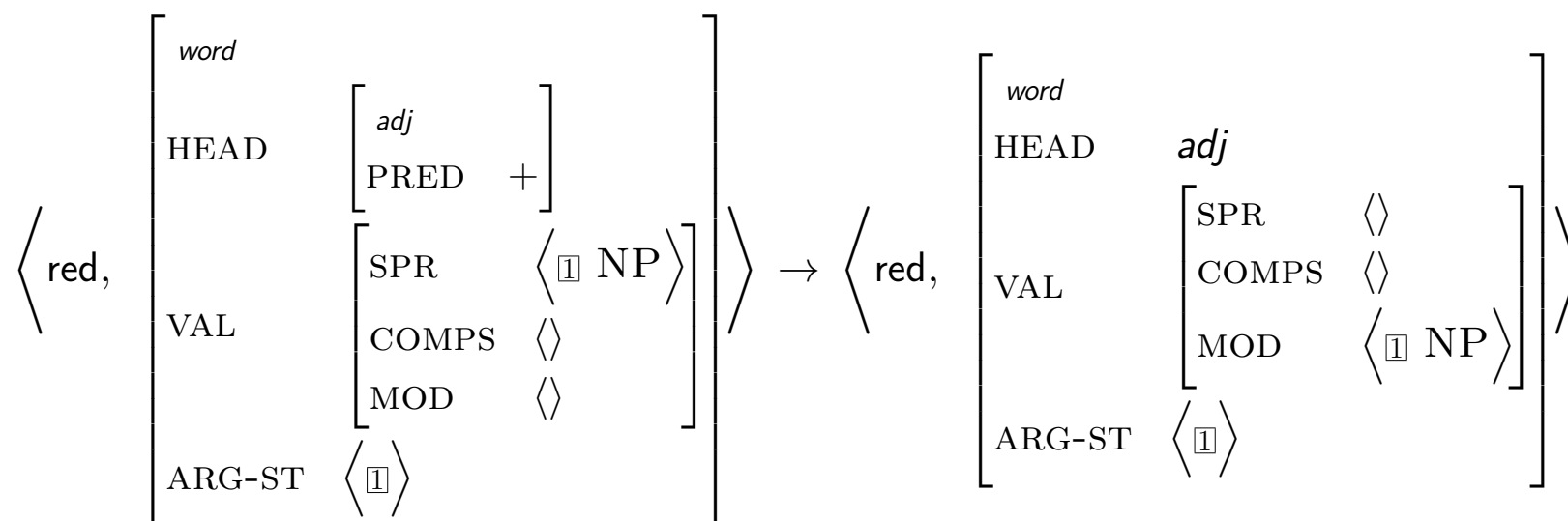
Overview

- Some examples of long-distance dependencies
- What is new and different about it
- Broad outlines of our approach
- Details of our approach
- Subject extraction
- Coordinate Structure Constraint

A Note on Adjectives

- Attributive adjectives are related to predicative adjectives by a lexical rule that co-indexes the first element of ARG-ST with MOD and sets SPR to an empty list:

The dog is red → *The red dog*



- We can't just co-index SPR and MOD. Why?
- The SPR adjectives need a subject-raising *be* to form a sentence. Why?

Long Distance Dependencies

Examples

- Grammatical:

(1) *Did you find something?*

(2) *Tell me you talked to someone!*

- wh-questions:

(3) *What did you find?*

(4) *Tell me who you talked to*

- relative clauses:

(5) *the item that I found*

(6) *the guy who(m) I talked to*

- Ungrammatical:

(7) **did you find*

(8) **you talked to*

- topicalization:

(9) *The manual, I can't find.*

(10) *Chris, you should talk to.*

- easy-adjectives:

(11) *My house is easy to find.*

(12) *Pat is hard to talk to.*

What these have in common

- There is a **gap**: nothing following *find* and *to*, even though both normally require objects.
- Something that fills the role of the element missing from the gap occurs at the beginning of the clause.
- We use topicalization and *easy*-adjectives to illustrate the phenomenon:

(13) *The manual_g, I can't find ______g*

(14) *Chris_g is easy to talk to ______g*

Gaps and their fillers can be far apart

(15) *The solution to this problem_g, Pat said that someone claimed you thought I would never find ______g.*

(16) *Chri_gs is easy to consider it impossible for anyone but a genius to try to talk to ______g.*

- Fillers often have syntactic properties associated with their gaps

(17) a. *Him_g, I haven't met ______g.*

b. **He_g, I haven't met ______g.*

(18) a. *The scissors_g, Pat told us _____ were missing_g.*

b. **The scissors_g, Pat told us _____ was missing_g.*

(19) a. *On Pat_g, you can rely ______g.*

b. **To Pat_g, you can rely ______g.*

- That's why we call them **long distance dependencies**

Other relevant facts

- Various languages show morphological marking on the verbs or complementizers of clauses between the filler and the gap.
- Psycholinguistic evidence indicates increased processing load in the region between filler and gap.

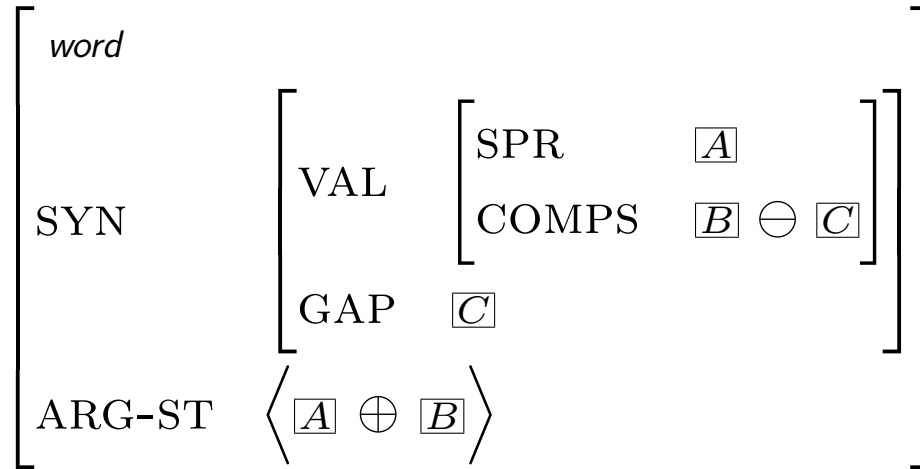
A Rough Sketch of Our Approach

- A feature GAP records information about a missing constituent.
- The GAP value is passed up the tree by a new principle.
- A new grammar rule allows us to expand S as a filler followed by another S whose GAP value matches the filler.
- Caveat: Making the details of this general idea work involves several complications.
- The core idea comes from Gazdar (1981)

The Feature gap

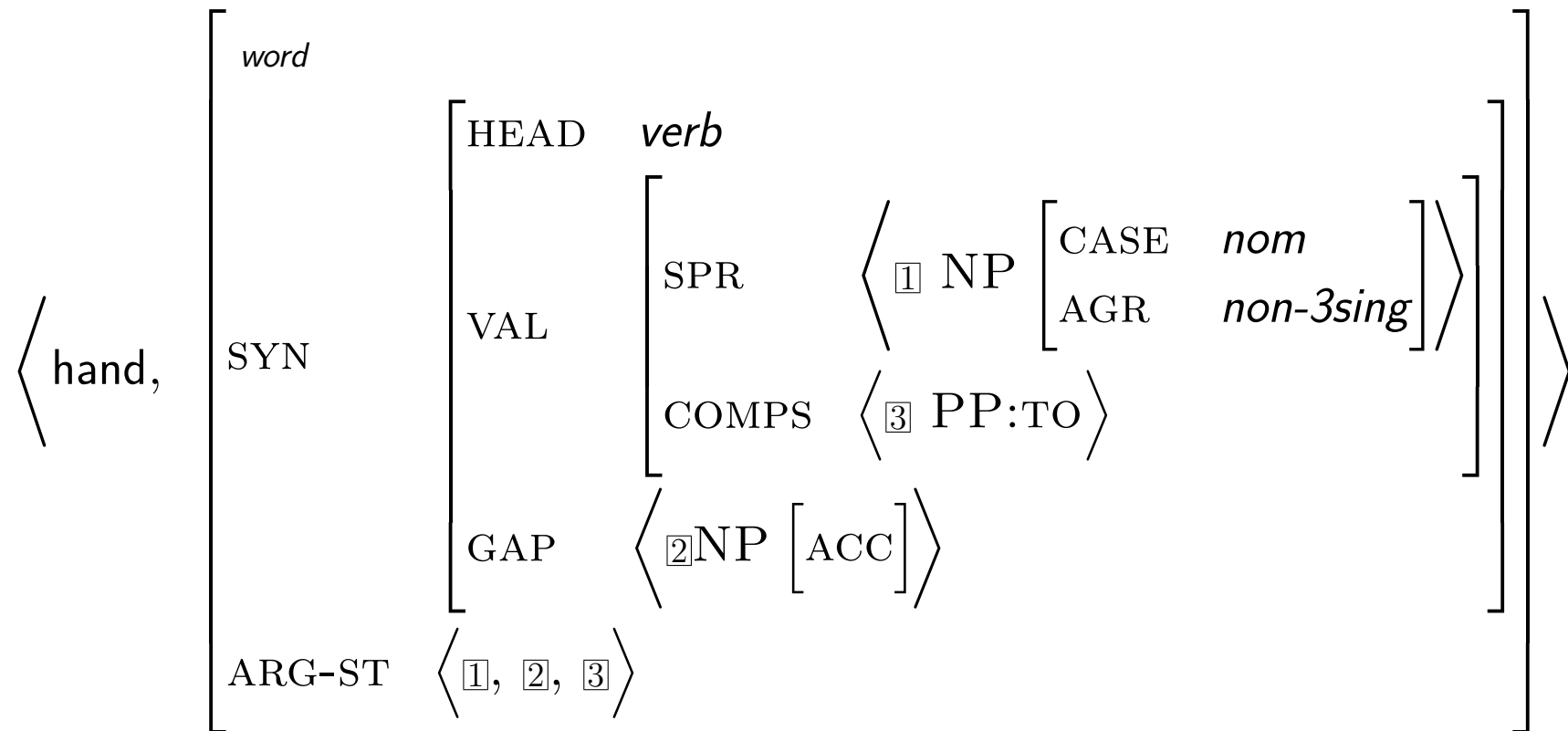
- Like valence features and ARG-ST, GAP's value is a list of feature structures (often empty). You can have multiple gaps.
- Subject gaps are introduced by a lexical rule.
- Non-subject gaps are introduced by revising the Argument Realization Principle.

The Revised ARP

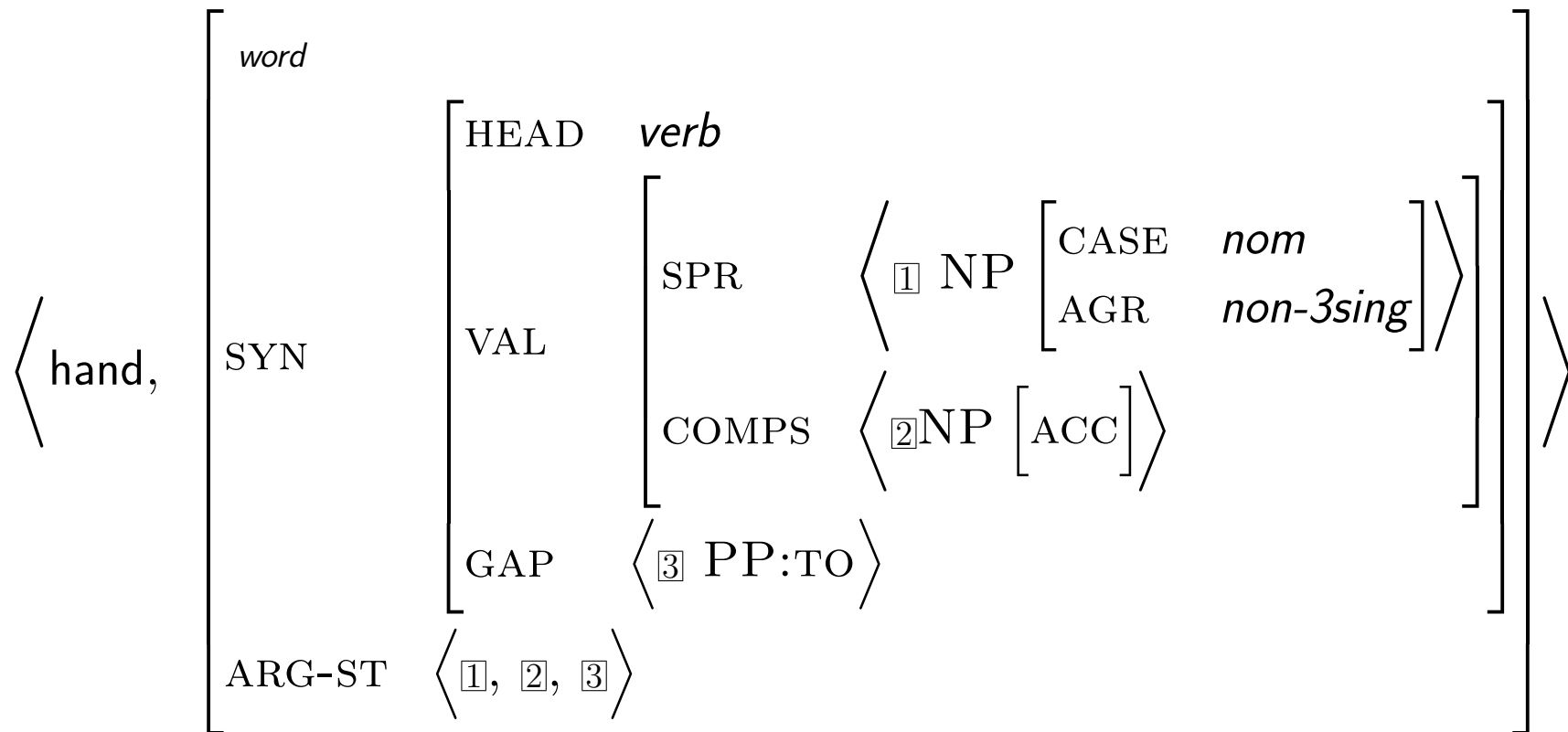


- $\ominus$ is a kind of list subtraction
 - it's not always defined (the sublist must exist on the main list)
 - when defined, it's not always unique
- The ARP now says the non-SPR arguments are distributed between COMPS and GAP.

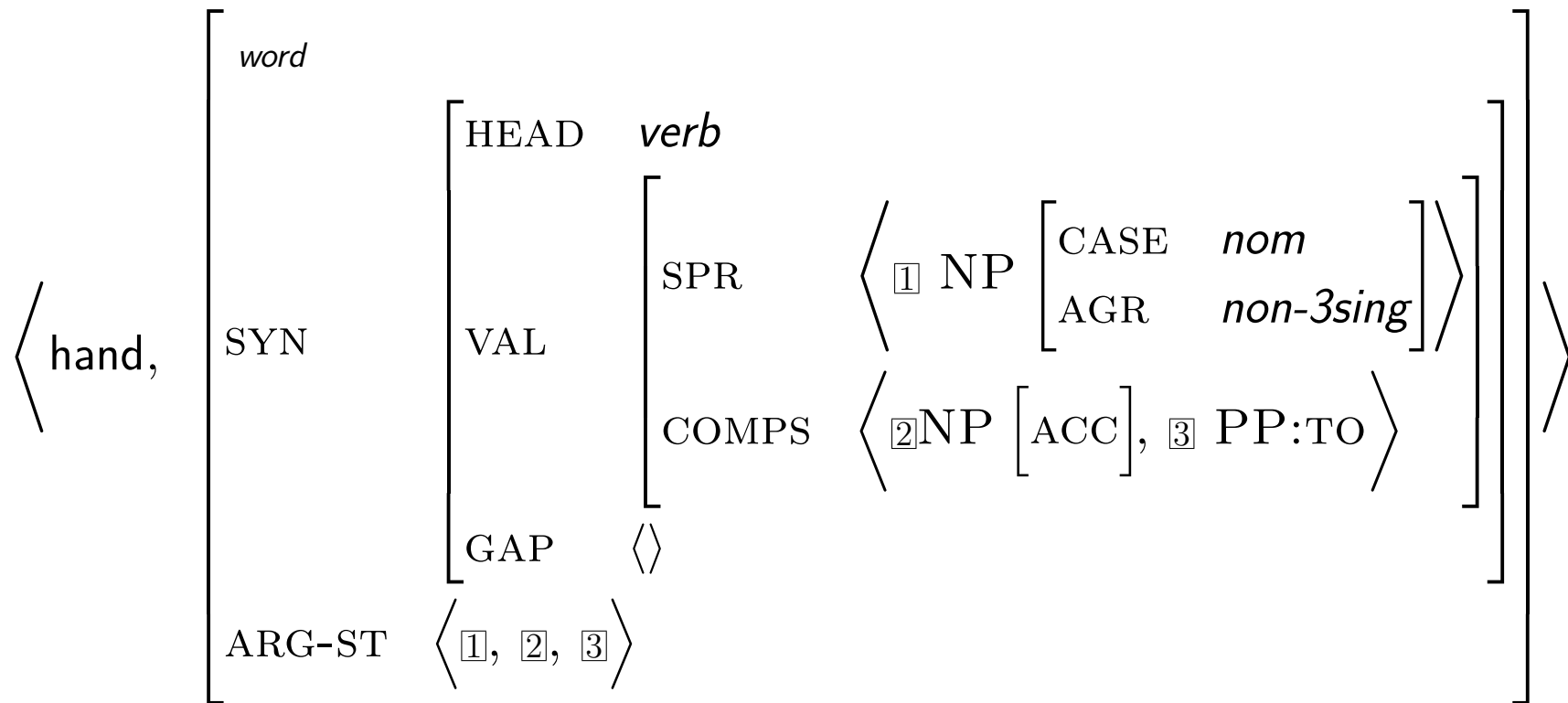
A Word with a Non-Empty gap Value



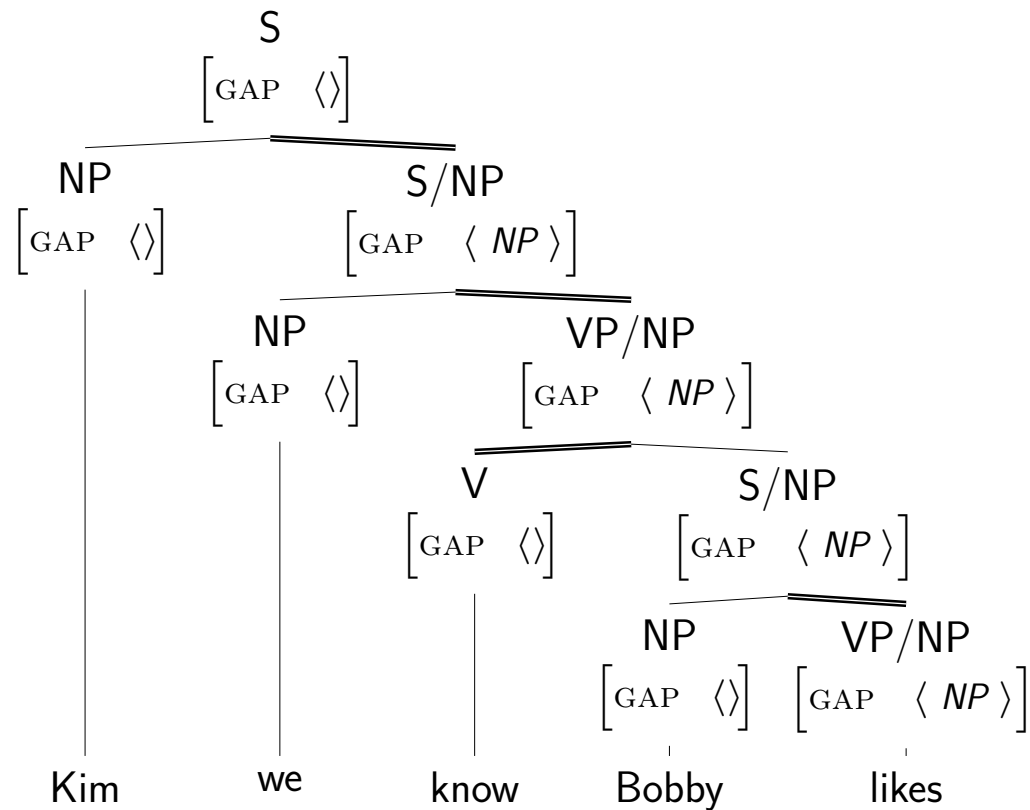
A Word with another Non-Empty gap Value



The same word with an Empty gap Value



How We Want GAP to Propagate



What GAP Propagation should doing

- Pass any GAP values from daughters up to their mothers,
... **except** when the filler is found.
- For topicalization, we can write the exception into the grammar rule
- For *easy*-adjectives, the NP that corresponds to the gap is the subject, which is introduced by the Head-Specifier Rule.
- Since specifiers are not generally gap fillers, we can't write the gap-filling into the HSR.

Our Solution to this Problem

- For *easy*-adjectives, we treat the adjective formally as the filler, marking its SPR value as coindexed with its GAP value.
- We use a feature STOP-GAP to trigger the emptying of the GAP list.
 - STOP-GAP stops gap propagation
 - *easy*-adjectives mark STOP-GAP lexically
 - a new grammar rule, the **Head-Filler Rule** contains STOP-GAP

The GAP Principle

A local subtree Φ satisfies the GAP Principle with respect to a headed rule if and only if Φ satisfies:

$$\begin{array}{c}
 \left[\text{GAP} \left(\boxed{A_1} \oplus \dots \oplus \boxed{A_n} \right) \ominus \boxed{A_0} \right] \\
 \hline
 \left[\text{GAP} \quad \boxed{A_1} \right] \quad \dots \quad \text{H} \left[\begin{array}{c} \text{GAP} \quad \boxed{A_i} \\ \text{STOP-GAP} \quad \boxed{A_0} \end{array} \right] \quad \dots \quad \left[\text{GAP} \quad \boxed{A_n} \right]
 \end{array}$$

- The GAP of the mother is the append of the GAPs of the daughters

... minus STOP-GAP on the head daughter

How does stop-gap work?

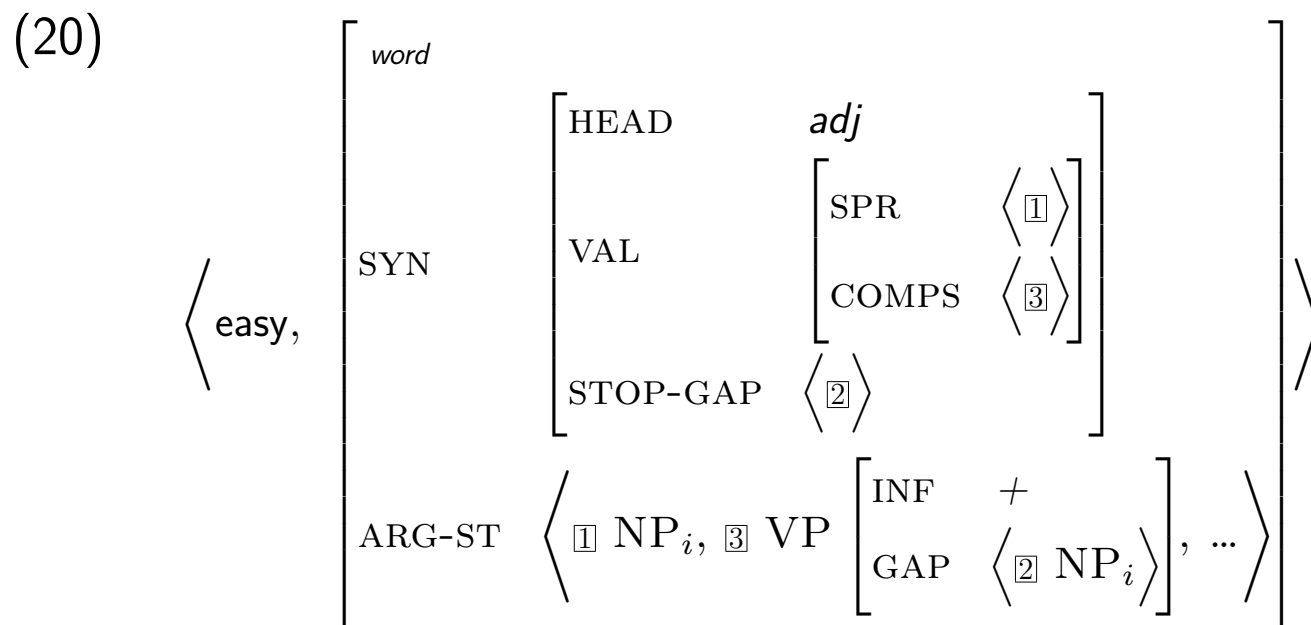
- STOP-GAP is empty almost everywhere
- When a gap is filled, STOP-GAP is nonempty, and its value is the same as the gap being filled.
- This blocks propagation of that GAP value, so gaps are only filled once.
- The nonempty STOP-GAP values come from two sources:
 - a stipulation in the Head-Filler Rule
 - lexical entries for *easy*-adjectives
- No principle propagates STOP-GAP

The Head-Filler Rule

$$[phrase] \rightarrow \boxed{1} [GAP \ \langle \rangle] \mathbf{H} \left[\begin{array}{ll} \text{HEAD} & \text{verb} \\ \text{VAL} & \left[\begin{array}{ll} \text{SPR} & \langle \rangle \\ \text{COMPS} & \langle \rangle \end{array} \right] \\ \text{GAP} & \langle \boxed{1} \rangle \\ \text{STOP-GAP} & \langle \boxed{1} \rangle \end{array} \right]$$

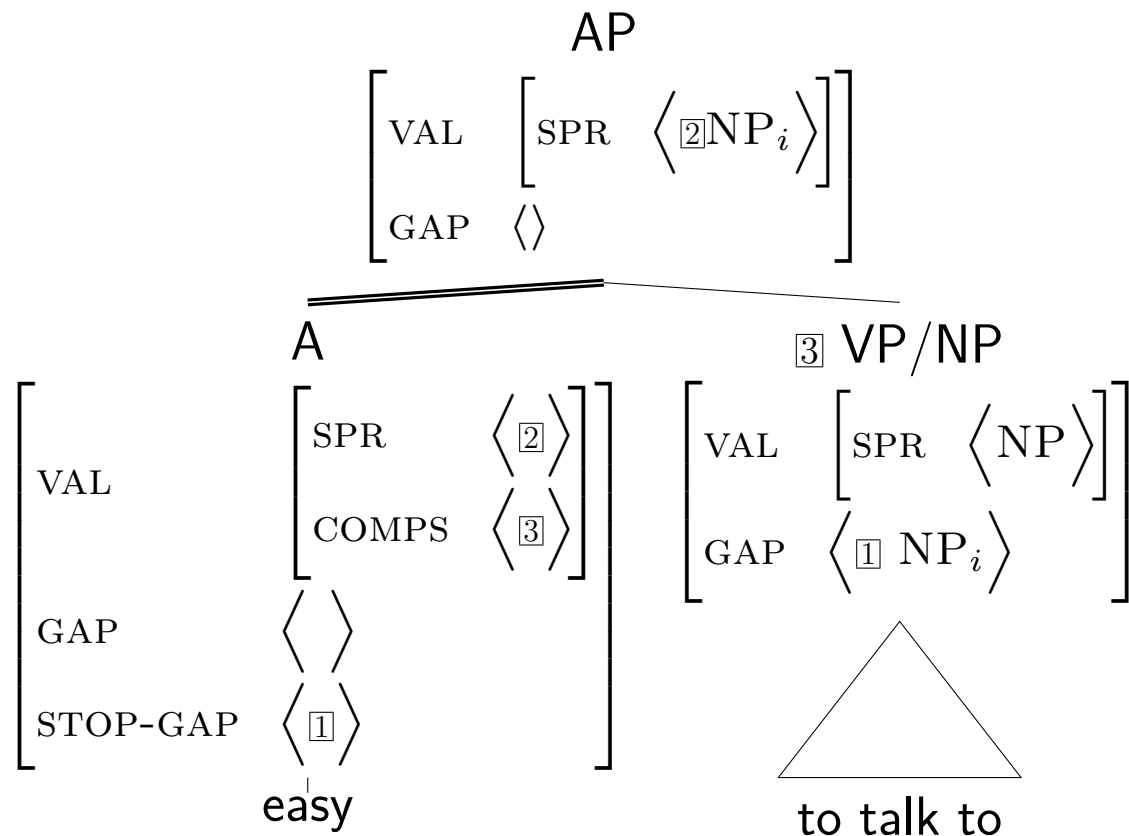
- This only covers gap filling in Ss
- The filler has to be identical to the GAP value
- The STOP-GAP value is also identical
- The GAP Principle ensures that the mother's GAP value is the empty list

Gap Filling with *easy*-Adjectives



- Because STOP-GAP and GAP have the same value, that value will be subtracted from the mother's GAP value.
- The first argument is coindexed with the GAP value, accounting for the interpretation of the subject as the filler.

A Tree for *easy to talk to* _____



stop-gap Housekeeping

- Lexical entries with nonempty STOP-GAP values (like *easy*) are rare, so STOP-GAP is by default empty in the lexicon.
- Head-Specifier and Head-Modifier rules need to say [STOP-GAP < >]
- Lexical rules preserve STOP-GAP values.

gap Housekeeping

Q The initial symbol must say [GAP <>]. Why?

A To block **Pat found* and **Chris talked to* as stand-alone sentences.

Q The Imperative Rule must propagate GAP values. Why?

A It's not a headed rule, so the effect of the GAP Principle must be replicated

A Imperatives can have gaps:

This book, put on the top shelf!

Sentences with Multiple Gaps

- Famous examples:

(21) *This violin_i, sonatas_j are easy to play ______j on ______i.*

(22) **Sonatas_j, this violin_i is easy to play ______j on ______i.*

- Our analysis gets this:
 - The subject of **easy** is coindexed with the first element of the GAP list.
 - The Head-Filler rule only allows one GAP remaining.
- There are languages that allow multiple gaps more generally.

Where We Are

- **filler-gap** structures:

(23) The solution to this problem, nobody understood _____

(24) That problem is easy to understand _____

- The feature GAP encodes information about missing constituents
- Modified ARP allows arguments that should be on the COMPS list to show up in the GAP list
- GAP values are passed up the tree by the GAP Principle

-
- The feature STOP-GAP signals where GAP passing should stop
 - The Head-Filler Rule matches a filler to a GAP and (via STOP-GAP) empties GAP
 - Lexical entries for *easy*-adjectives require a gap in the complement, coindex the subject with the gap, and (via STOP-GAP) empty GAP on the mother

More Phenomena filler ...

- Sentences with subject gaps
- Gaps in coordinate constructions

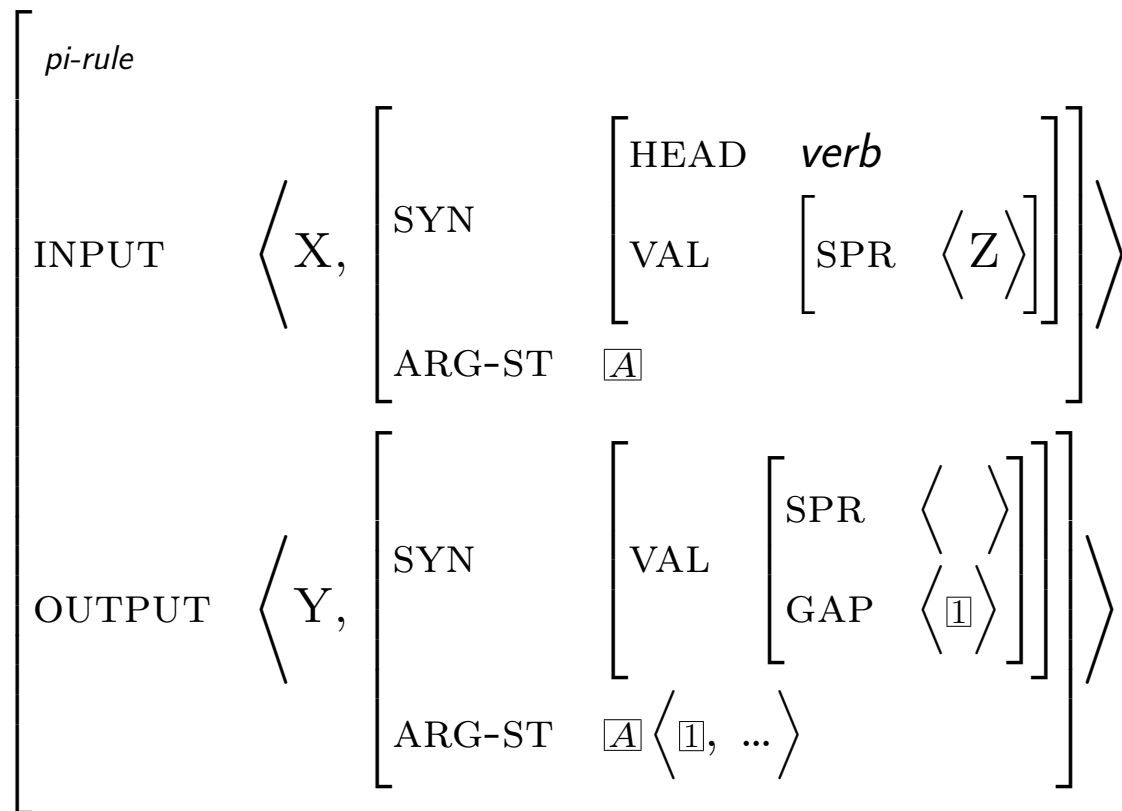
Subject Gaps

- The ARP revision only allowed missing complements.
- But gaps occur in subject position, too:

(25) *This problem, everyone thought _____ was too easy.*

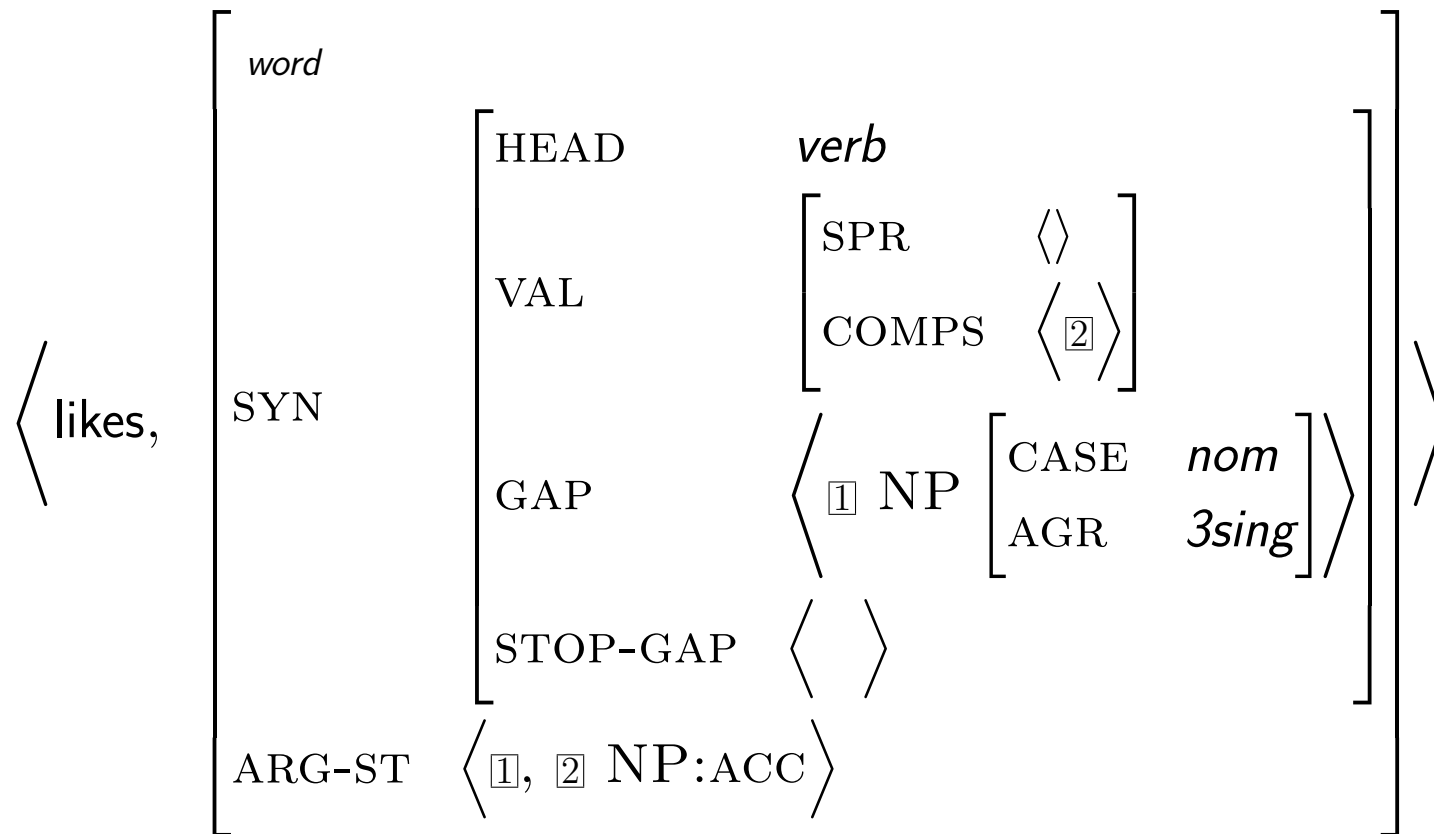
- We handle these via a lexical rule that, in effect, moves the contents of the SPR list into the GAP list

The Subject Extraction Lexical Rule



- NB: This says nothing about the phonology, because the default for pi-rules is to leave the phonology unchanged.

A Lexical Sequence This Licenses



- Note that the ARP is satisfied

Island Constraints

- There are configurations that block filler-gap dependencies, sometimes called **islands**
- Trying to explain them has been a central topic of syntactic research since the mid 1960s
- We'll look at just one, Ross's so-called **Coordinate Structure Constraint**
- Loose statement of the constraint: a constituent outside a coordinate structure cannot be the filler for a gap inside the coordinate structure.

Coordinate Structure Constraint Examples

- (26) **This problem, nobody finished the extra credit and _____*
- (27) **This problem, nobody finished _____ and the extra credit.*
- (28) **This problem, nobody finished _____ and started the extra credit.*
- (29) **This problem, nobody started the extra credit and finished _____*
- (30) *This problem, everybody started _____ and nobody finished _____*

- In a coordinate structure,
 - no conjunct can be a gap (**conjunct constraint**)
 - no gap can be contained in a conjunct if its filler is outside of that conjunct (element constraint)
 - ... unless each conjunct has a gap that is paired with the same filler (**across-the-board exception**)

These observations cry out for explanation

- In our analysis, the conjunct constraint is an immediate consequence: individual conjuncts are not on the ARG-ST list of any word, so they can't be put on the GAP list
- The element constraint and ATB exception suggest that GAP is one of those features (along with VAL and FORM) that must agree across conjuncts.
- Note: There is no ATB exception to the conjunct constraint.

(31) **This problem, you can compare only _____ and _____.*

Our Coordination Rule, so far

$$\begin{bmatrix} \text{VAL} & \boxed{0} \\ \text{IND} & s_0 \end{bmatrix} \rightarrow \begin{bmatrix} \text{VAL} & \boxed{0} \\ \text{IND} & s_1 \end{bmatrix} \cdots \begin{bmatrix} \text{VAL} & \boxed{0} \\ \text{IND} & s_{n-1} \end{bmatrix} \begin{bmatrix} \text{HEAD} & \textit{conj} \\ \text{IND} & s_0 \\ \text{RESTR} & \langle \begin{bmatrix} \text{ARGS} & \langle s_1, \dots, s_{n-1}, s_n \rangle \end{bmatrix} \rangle \end{bmatrix} \begin{bmatrix} \text{VAL} & \boxed{0} \\ \text{IND} & s_n \end{bmatrix}$$

- Recall that we have tinkered with what must agree across conjuncts at various times.
- Now we'll add GAP to the things that conjuncts must share

Our Final Coordination Rule

$$\begin{bmatrix} \text{VAL} & \boxed{0} \\ \text{IND} & s_0 \\ \text{GAP} & \boxed{A} \end{bmatrix} \rightarrow \begin{bmatrix} \text{VAL} & \boxed{0} \\ \text{IND} & s_1 \\ \text{GAP} & \boxed{A} \end{bmatrix} \dots \begin{bmatrix} \text{VAL} & \boxed{0} \\ \text{IND} & s_{n-1} \\ \text{GAP} & \boxed{A} \end{bmatrix} \begin{bmatrix} \text{HEAD} & conj \\ \text{IND} & s_0 \\ \text{RESTR} & \langle \begin{bmatrix} \text{ARGS} & \langle s_1, \dots, s_{n-1}, s_n \rangle \end{bmatrix} \rangle \end{bmatrix} \begin{bmatrix} \text{VAL} & \boxed{0} \\ \text{IND} & s_n \\ \text{GAP} & \boxed{A} \end{bmatrix}$$

- We've just added GAP to all the conjuncts and the mother.
- This makes the conjuncts all have the same gap (if any)
- Why do we need it on the mother?

Closing Remarks on LDDs

- This is a huge topic; we've only scratched the surface
- There are many more kinds of LDDs, which would require additional grammar rules
- There are also more island constraints, which also need to be explained
- Our account of the coordinate structure constraint (based on ideas of Gazdar) is a step in the right direction, but it would be even better to explain why certain features must agree across conjuncts.

Overview of LDD

- Some examples of the phenomenon
- What is new and different about it
- Broad outlines of our approach
- Details of our approach
- Subject extraction
- Coordinate Structure Constraint

P0: Semantics are *easy*

Add the semantics to the lexeme *easy* given on slide 20.

Then give the full rels list for the top node (i.e. the whole sentence) for (32) and (33). What is the deep subject of *easy* in each sentence?

(32) *My house is easy to find.*

(33) *Pat is easy to talk to.*

P1: A Tree with a Gap

Draw a tree for (34). Use abbreviations for the node labels, and show the value of GAP on all nodes. Show the value of STOP-GAP on any node where it is non-empty.

(34) *This baby, I know that they handed a toy to _____*

P2: Blocking Filled Gaps

Examples (i) and (ii) are well-formed, but example (iii) is ungrammatical:

- (i) *Pat thinks that I rely on some sort of trick.*
- (ii) *This mnemonic, Pat thinks that I rely on.*
- (iii) **This mnemonic, Pat thinks that I rely on some sort of trick.*

Explain in detail why the mechanisms that license (i) and (ii) do not also permit (iii).

P3: Subject Gaps

This problem is to make sure you understand how our analysis accounts for examples like (35).

- (35) i. *Which candidates do you think like oysters on the half-shell?*
 ii. *That candidate, I think likes oysters on the half-shell.*

- A. Sketch the family of lexical sequences for *likes* that is the input to the Subject Extraction Lexical Rule.
- B. Sketch the family of lexical sequences for *likes* that is the corresponding output of the Subject Extraction Lexical Rule.
- C. Sketch the tree for the sentence in (35ii). Use abbreviations for node labels, but show the value of GAP on all nodes and the value of STOP-GAP on any node

where it is non-empty. You may abbreviate the structure over the NP *oysters on the half-shell* with a triangle.

D. Does our analysis correctly predict the contrast between (35ii) and 36?

(36) **Those candidates, I think likes oysters on the half-shell.*

Explain why or why not.

Acknowledgments and References

- Course design and slides borrow heavily from Emily Bender's course: *Linguistics 566: Introduction to Syntax for Computational Linguistics*
<http://courses.washington.edu/ling566>



References

- Gerald Gazdar. 1981. Unbounded dependencies and coordinate structure. *Linguistic Inquiry*, 12:155–184.
- Ivan A. Sag, Tom Wasow, and Emily Bender. 2003. *Syntactic Theory: A Formal Introduction*. CSLI Publications, Stanford, second edition.